Lab 04 | Distributed Leader Election

BÁO CÁO KIẾN TRÚC | Go - TCP sockets - Bully Algorithm

1. Mục tiêu và giả định

Mô phỏng năm tiến trình P1-P5 bằng goroutine trong một chương trình Go. Mỗi node có TCP listener riêng trên localhost, cổng trống được hệ điều hành cấp. P5 là coordinator ban đầu. Bộ mô phỏng dừng P5; P1-P4 phải tự phát hiện lỗi và thống nhất P4 là leader mới, không được bộ mô phỏng gán kết quả.

Membership và ID cố định; ID lớn hơn có ưu tiên cao hơn. Phạm vi là crash-stop một node trên mạng localhost ổn định, không có khôi phục node hoặc phân hoạch mạng. Goroutine được Go runtime lập lịch lên các OS thread; đây không phải năm chương trình OS riêng biệt.

2. Phân chia thành phần

main.go đọc tham số, mở log và xử lý Ctrl+C. simulation.go tạo cluster, tiêm lỗi, kiểm tra ổn định và dọn tài nguyên. protocol.go định nghĩa Message, Timing và ba trạng thái. transport.go thực hiện TCP request/response. node.go chứa logic Bully. node_test.go kiểm tra socket và chuyển trạng thái.

Mỗi node có một goroutine run() duy nhất sửa trạng thái bầu cử. Goroutine TCP nhận JSON rồi chuyển request vào inbox; worker gửi TCP chuyển kết quả vào results. Channel chỉ dùng bên trong một node. Giữa hai node, ELECTION và COORDINATOR đều đi qua TCP thật. Peer map được sao chép và chỉ đọc.

3. Giao thức truyền thông

Một TCP connection mang một request JSON và một response JSON rồi đóng. Message gồm type và from. PING được leader trả ALIVE; ELECTION gửi tới các ID cao hơn và nhận OK; COORDINATOR công bố người thắng và nhận OK xác nhận. Request sai loại hoặc ID không hợp lệ nhận REJECT. Client xác nhận cả loại phản hồi lẫn ID của peer.

RPC có hạn kết nối 200 ms và deadline trao đổi 200 ms sau khi kết nối. Follower hỏi leader mỗi 300 ms. Lỗi kết nối, hết hạn hoặc phản hồi không hợp lệ đều kích hoạt nghi ngờ lỗi. Dòng log Timeout bao gồm cả connection refused khi P5 đã đóng endpoint. Heartbeat thành công không in để trace gọn.

4. Đồng bộ và vòng đời

Gửi TCP chạy ở worker riêng để node không chặn vòng lặp trong lúc chờ peer. Biến round đánh dấu phiên I/O nội bộ; kết quả khác round hiện tại bị loại. publishedLeader dùng atomic để main đọc; logger dùng mutex để không chen dòng.

Stop đóng channel done, đóng listener và chờ WaitGroup. Go không hỗ trợ kill goroutine tùy ý; dừng hợp tác và đóng endpoint mô phỏng crash-stop. Socket có deadline để không treo khi đọc. sync.Once cho phép Stop lặp an toàn. P1-P4 không nhận tín hiệu dừng của P5, chỉ quan sát lỗi qua TCP.

Bully, kiểm chứng và giới hạn

5. Máy trạng thái và lựa chọn leader

IDLE: follower gửi heartbeat; leader trả ALIVE. Heartbeat lỗi hoặc nhận ELECTION từ ID nhỏ hơn sẽ gọi startElection. Node tăng round, đặt leader = 0 (chưa biết), sang WAIT_OK và gửi ELECTION tới mọi ID cao hơn. Node đang bầu vẫn trả OK cho node thấp hơn nhưng không mở vòng bầu song song cho chính nó.

WAIT_OK: chờ tối đa 600 ms. Không có OK thì becomeLeader đặt leader bằng ID của mình, về IDLE và gửi COORDINATOR tới mọi ID thấp hơn. Có OK thì chuyển WAIT_COORDINATOR, chờ tối đa 1.500 ms. Hết hạn mà chưa có coordinator thì bầu lại; tránh kẹt khi node cao hơn trả OK rồi crash.

COORDINATOR từ ID cao hơn đưa node về IDLE, cập nhật leader, tăng round để vô hiệu hóa kết quả I/O cũ và lên lịch heartbeat mới. Tuyên bố thắng từ ID thấp hơn bị từ chối và node cao hơn khởi động bầu cử.

6. Fault injection và trace kỳ vọng

simulate khởi động năm node với P5 làm leader, chờ crash-after (mặc định 1 giây), rồi chỉ Stop P5. Một hoặc nhiều follower phát hiện P5 không trả ALIVE. Ví dụ P2 gửi ELECTION đến P3, P4, P5; P3/P4 trả OK và tự bầu. P4 không nhận OK từ P5 nên thắng, gửi COORDINATOR đến P1-P3.

Log thể hiện địa chỉ node, leader dừng, phát hiện timeout, gửi/nhận ELECTION, OK, chiến thắng và nhận COORDINATOR. Console đồng thời được lưu trace.log. Thứ tự xen kẽ phụ thuộc goroutine; không hard-code P2 khởi xướng. File trace đi kèm được tạo bằng chạy thực tế.

7. Xác minh và kiểm thử

Main chờ tối đa 10 giây để P1-P4 đều nhận leader = 4, rồi yêu cầu kết quả giữ ổn định liên tục qua các lần lấy mẫu 20 ms trong 600 ms. Nếu có node đổi leader, khoảng ổn định được tính lại. Thành công ghi VERIFY/PASS; không hội tụ, context hủy hoặc lỗi ghi log trả lỗi và exit code 1.

Kiểm thử gồm kịch bản crash thật, leader khỏe, socket đã đóng, server nhận TCP nhưng không trả lời, retry khi thiếu COORDINATOR, bỏ OK cũ, từ chối ID không hợp lệ/thấp hơn, hủy mô phỏng và chạy lặp. Lệnh kiểm tra: go test -v ./..., go vet ./..., go test -race ./... khi môi trường hỗ trợ.

8. Giới hạn và rubric

Timeout chỉ là nghi ngờ lỗi. Bully có thể xuất hiện nhiều leader tạm thời khi mạng phân hoạch; project không cung cấp đồng thuận chịu lỗi mạng tùy ý. Không có xác thực, persistent state hoặc node recovery. Chi phí ELECTION có thể $O(n^2)$. COORDINATOR gửi một lần; follower thiếu thông báo sẽ retry. round không phải term phân tán và không loại mọi thông báo mạng cũ.

Rubric: logic Bully 40% ở máy trạng thái; phát hiện lỗi 30% ở heartbeat và deadline; trace 20% ở console/file và VERIFY; báo cáo 10% ở hai trang này. README giải thích chi tiết từng file, hàm và ví dụ. Chạy demo: go run .; biên dịch Windows: go build -o leader-election.exe .